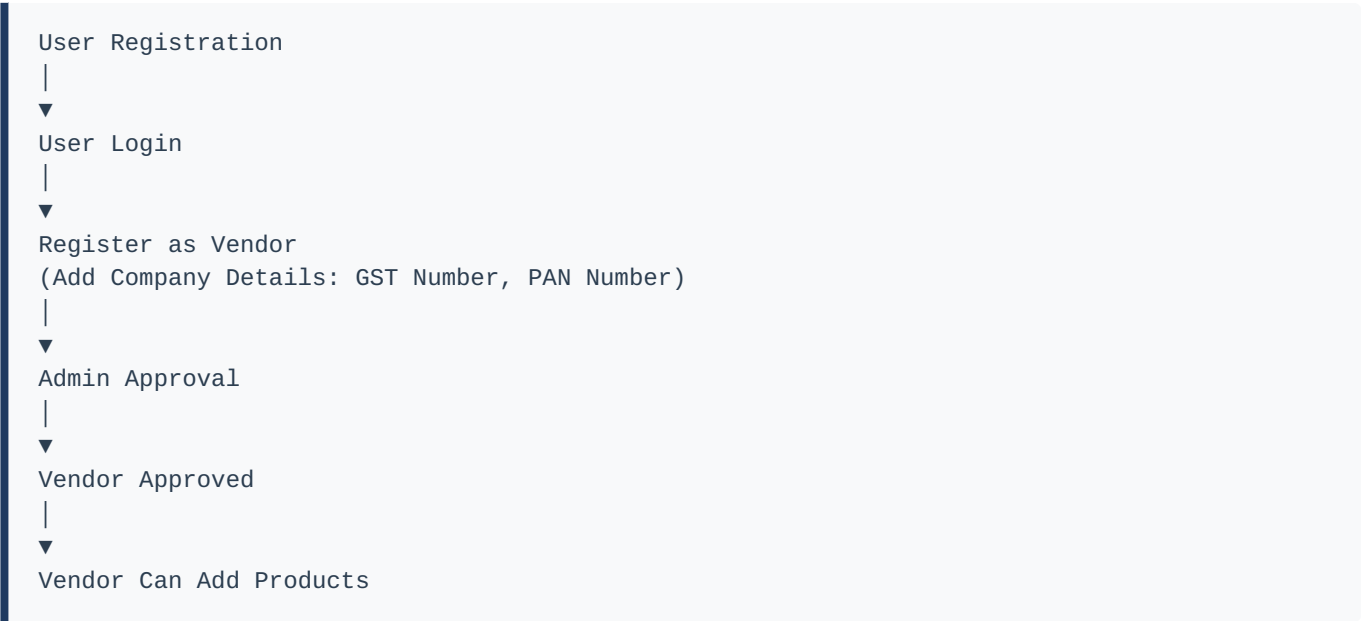


E-Commerce System Architecture

COMPREHENSIVE WORKFLOW & SETTLEMENT ENGINE SPECIFICATION

A. Vendor Management Workflow

Process Flow



Process Description

1. A user first registers and logs into the system seamlessly.
2. The authenticated user can apply for vendor registration by securely providing necessary corporate company details, including the corporate **GST Number** and **PAN Number**.
3. The system administrator verifies the credentials and approves or rejects the vendor onboarding request.
4. Once officially approved, the user's status is upgraded, granting full access to integrated product management features.

B. Product Management Workflow

Process Flow

```
Vendor Creates Product
(Common Details: Name, Description)
|
▼
Add Product Variants
(Quantity, Price, Specifications)
|
▼
Publish Product
|
▼
Customers Browse Products
(Filter + Pagination + Dynamic URL Endpoints)
```

Process Description

1. Vendors initialize products by establishing common architectural information such as localized name, categorization, and system description.
2. Each primary product can support multiple distinct variants, with standalone control over its own inventory quantity, base pricing, and specific product attributes (e.g., color, size).
3. Upon publishing the inventory, the items instantly become visible and available for active customer procurement.
4. Customers can efficiently search, filter, and browse products utilizing clean pagination structures and dynamic query endpoints.

C. Order & Transaction Management

1. Add to Cart Workflow

```
Customer Selects Product Variant
|
▼
Stock Availability Check
|
▼
Create/Update Cart Item
```

Functional Rationale:

- Active shopping carts, available stock data, variant prices, and logistical shipments are systematically structured and maintained exclusively at the **Product Variant** level.

- If a selected variant already exists within the user's active session cart, the corresponding item quantity is updated inside the database.
- Otherwise, a completely isolated and new Cart Item record is generated.

2. Place Order Workflow

```

Revalidate Stock
|
▼
Validate Discount Code
|
▼
Calculate Shipping Fee (Per Vendor)
|
▼
Calculate Total Amount (Gross Total + Tax + Shipping - Discount)
|
▼
Create Order (Pending)
|
▼
Create Order Items
|
▼
Create Payment (Pending)
|
▼
Create Shipment (Pending)
|
▼
Reserve Stock (Update Reserved Stock Column)

```

Execution Architecture Specifications

1. **Stock Verification:** Immediate real-time stock quantities are completely re-verified to block concurrency race conditions.
2. **Discount Processing:** Applicable digital discount coupons are parsed and validated against promotion constraints.
3. **Logistics Fees:** Shipping charges are derived independently for each vendor, as vendors directly handle their respective physical logistics fulfillment loops.
4. **Financial Computations:** The final checkout transactional calculation is accurately executed via the following programmatic logic:

$$\text{Total Amount} = \text{Gross Total} + \text{Tax} + \text{Shipping Fee} - \text{Discount}$$

1. **Entity Generation:** The database securely processes and instantiates the following explicit structures:
 - **Order** → Set to initial state *Pending*
 - **Order Items** → Granular mapping of discrete variants

- **Payment** → Set to initial state *Pending*
- **Shipment** → Set to initial state *Pending*

2. **Stock Reservation Engine:** Required inventory counts are systematically moved to a dedicated Reserved Stock attribute column.
3. **TTL Automated Release:** Reserved stock is automatically released back to public availability after exactly 10 minutes if the corresponding payment gateway lifecycle is not successfully finalized.
4. **Data Integrity:** All operations are completely wrapped and executed inside a strict localized database transaction block to guarantee absolute data consistency.

Payment Architecture Update

Stripe Test Environment is used natively for processing all transactions. Input parameters require a valid Order ID and an applicable Payment Method Token (e.g., pm_card_visa).

3. Payment Lifecycle Workflow

```
Customer Initiates Payment (orderId + paymentMethod)
|
▼
Stripe Payment Execution (Test Environment)
|
▼
Payment Successful
|
├── ────────────▶ Order Status = Confirmed
├── ────────────▶ Payment Status = Confirmed
├── ────────────▶ Shipment Status = Initiated
├── ────────────▶ Reserved Stock Released
├── ────────────▶ Actual Stock Decrement
├── ────────────▶ Email Notification Sent
└── ────────────▶ Vendor Settlement Triggered (Evaluate Distribution Type)
```

Post-Payment State Changes

Upon receiving a successful synchronous confirmation from the Stripe gateway API, the core system initiates the following transactional actions:

- The global Order status transitions immediately to **Confirmed**.
- The linked Payment token status mutates securely to **Confirmed**.
- The associated individual logistical Shipment records transition to **Initiated**.
- The temporary Reserved Stock state allocation is released.
- The physical warehouse Actual Stock records are decremented.
- An asynchronous outbound order confirmation email is fired via the SMTP Service.
- The core **Discount Distribution and Settlement Engine** is asynchronously invoked.

4. Post-Purchase Review Workflow

```
Customer Purchases Product → Order Completed → Customer Can Review Product
```

- Customers are allowed to review only the specific structural products they have completely purchased and received.
- Reviews are explicitly associated with the generalized core **Product** entity rather than granular **Product Variants**. This design avoids duplicate reviews across variants and improves discoverability.

Entity Relationship & Architecture Blueprint

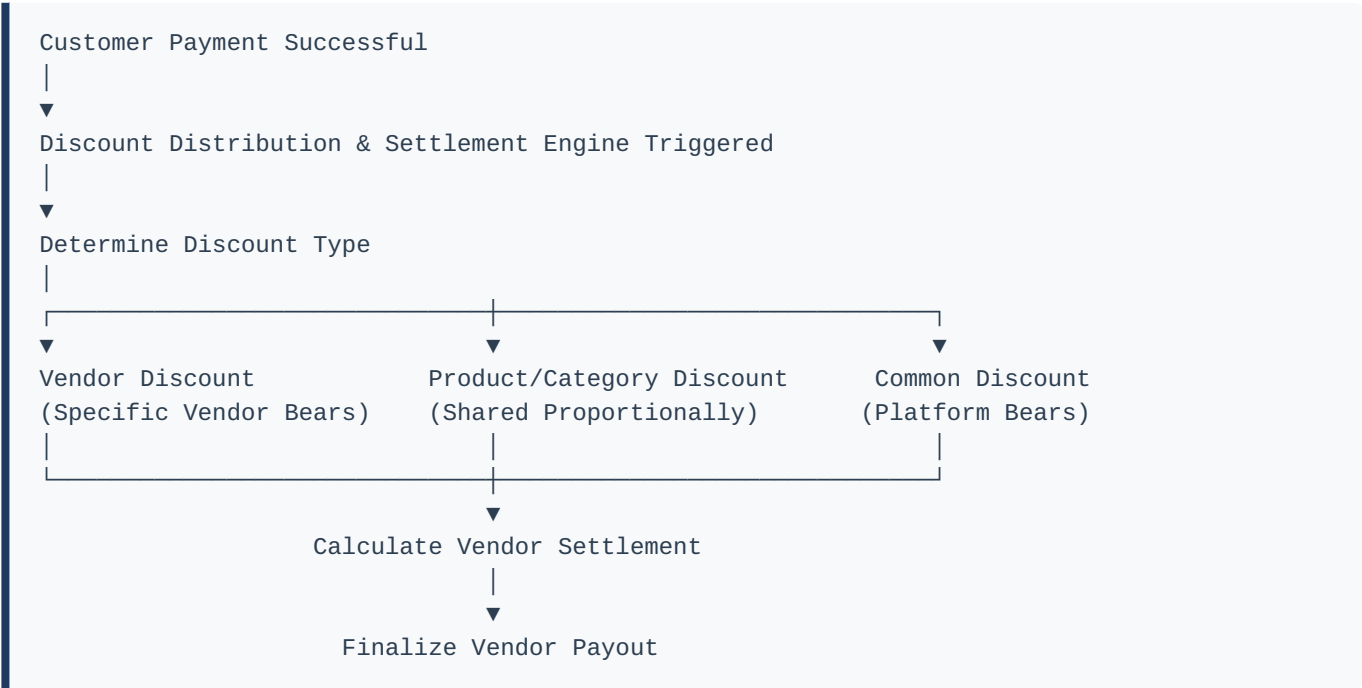


Architectural Rationale

Cart, Wishlist, Order Items, Stock, Price, and Shipment are strictly isolated to **Product Variants**, because these core parameters inherently fluctuate depending on specialized attributes like size, color, or specifications. Linking reviews directly to the core **Product** entity mirrors highly scalable marketplace models (such as Amazon), consolidating feedback mechanisms and avoiding fragmentation.

D. Discount Distribution & Settlement Engine

Engine Architecture



Detailed Settlement Models

1. Vendor-Level Discount

Vendor-specific promotional campaigns are entirely sponsored by that unique vendor. Therefore, the vendor offering the voucher code single-handedly bears the full cost of the discount.

Settlement Amount = Base Product Price – Vendor Discount

Example: ₹1,000 Price – ₹100 Discount = **₹900 Vendor Settlement**

2. Product-Level or Category-Level Discount

These markdown campaigns apply evenly to products belonging to specified system groups. If an order spans products from multiple vendors, the discount liability is distributed proportionally matching their respective order weight values.

Vendor Entity	Gross Product Value	Proportional Contribution	Final Vendor Payout
Vendor A	₹2,000	₹200	₹1,800
Vendor B	₹1,000	₹100	₹900
Total Summary	₹3,000	₹300 (Total Order Discount)	₹2,700 (Combined Payout)

3. Common (Platform-Wide) Discount

Common discounts represent global sitewide promotional offerings engineered and sponsored directly by the platform administration team. Marketplace vendors remain completely insulated from these margin markdowns and receive full financial reimbursement for their sold product listings.

- **Gross System Order Value:** ₹1,000
- **Platform Sponsored Discount:** ₹100
- **Total Customer Paid Amount:** ₹900
- **Final Vendor Disbursal Payout: ₹1,000** (Platform bears the ₹100 margin loss internally)

Core Technology Stack Implementation

Backend Framework	ASP.NET Core Web API	Payment Provider	Stripe API (Test System)
Data Access	Entity Framework Core	Mailing Service	SMTP Automated Email Engine
Database Engine	Microsoft SQL Server	Transactional Model	EF Core ACID Transactions

Engine Separation Benefits & Rationale

The **Discount Distribution and Settlement Engine** is intentionally separated from synchronous order creation pipelines and inline checkout execution blocks to secure clean modularity, domain boundaries, and long-term codebase maintainability. By isolating settlement logic into an asynchronous background service loop, the architecture safely provides robust scalability for multi-vendor marketplace scenarios, facilitates error-free financial auditing, and allows deployment of complex promotional campaigns mirroring enterprise systems like Amazon and Flipkart.